

Aplicação de Inteligência Artificial na Identificação de Vulnerabilidade em Redes Locais com Integração a Firewall

João Vitor Ielen, Vinicius Mota Favaro

Orientador: Michel Cesar dos Santos

Resumo.

Este trabalho propõe como objetivo explorar a aplicação de inteligência artificial no campo da segurança cibernética, com o foco na automação de processos de identificação de vulnerabilidades em redes locais. Essa proposta visa exibir o aumento da eficiência na detecção de riscos e na prevenção de ameaças, promovendo uma maior precisão na proteção de sistemas. A abordagem envolve a integração de mecanismos de exibição e conexão com ferramentas de segurança, como Firewalls, permitindo ações automatizadas de mitigação. O projeto é conduzido com base em metodologias ágeis, garantindo a flexibilidade, ciclos de entrega contínuos e alinhamento com os princípios da segurança da informação.

Palavras chave: segurança de redes, inteligência artificial, Firewall, Pentest, resposta automatizada.

Application of Artificial Intelligence in Identifying Vulnerabilities in Local Area Networks with Firewall Integration

Abstract.

This work aims to explore the application of Artificial Intelligence (AI) in the field of cybersecurity, focusing on automating processes for identifying vulnerabilities in local networks. The proposal seeks to demonstrate increased efficiency in risk detection and threat prevention, promoting greater accuracy in system protection. The approach involves the integration of display mechanisms and connection with security tools, such as firewalls, enabling automated mitigation actions. The project is conducted based on agile methodologies, ensuring flexibility, continuous delivery cycles, and alignment with information security principles.

Key-words: network security, artificial intelligence, firewall, pentest, automated response.

1. Introdução.

A digitalização dos processos empresariais ampliou a eficiência operacional, mas também tornou as redes corporativas mais expostas a ataques cibernéticos. Ambientes sem segmentação adequada, dispositivos desatualizados e ausência de monitoramento contínuo elevam significativamente o risco de incidentes. Relatórios recentes apontam crescimento expressivo nas ameaças: o Midyear Threat Report 2025 (Forescout, 2025) registrou aumentos de até 72% em tentativas de invasão em setores como varejo e finanças, enquanto a Varonis Threat Labs (2025) identificou crescimento superior a 90% nos ataques de ransomware no período. Esses dados evidenciam que a segurança de rede permanece como um dos pilares essenciais para continuidade dos negócios e proteção de dados.

Nesse cenário, soluções capazes de identificar vulnerabilidades de forma autônoma tornam-se fundamentais. Este trabalho propõe o desenvolvimento de um sistema de detecção inteligente baseado em IA, capaz de analisar redes locais, identificar falhas e interagir com firewalls para mitigar riscos automaticamente, reduzindo a dependência de intervenção humana e fortalecendo a resposta a incidentes.

1.1 Formulação de Tema

O tema central deste trabalho é o uso de Inteligência Artificial aplicada à segurança cibernética, com foco na automação das etapas de detecção, análise e mitigação de vulnerabilidades em redes locais. A solução proposta integra Nmap, Python e modelos de linguagem executados localmente via Ollama (Llama 3.2), formando um pipeline capaz de identificar portas e serviços inseguros, contextualizar riscos com base em bases especializadas (CVE/NVD, OWASP, NIST) e recomendar ações de mitigação alinhadas a boas práticas como LGPD, ISO 27001 e PCI-DSS.

O sistema automatiza tarefas tradicionalmente manuais — classificação de severidade, mapeamento de CVEs, construção de cenários de ataque e verificação

de conformidade — e gera relatórios estruturados em JSON, Markdown e HTML. A execução totalmente local assegura privacidade dos dados e evita exposição da topologia para serviços em nuvem, aspecto crítico em auditorias corporativas. Ao combinar detecção técnica com análise contextual feita pelo LLM, a proposta supera limitações de scanners tradicionais, oferecendo uma visão mais profunda e precisa sobre riscos e possíveis vetores de ataque.

1.2 Formulação do Problema

Como a aplicação de técnicas de inteligência artificial pode contribuir para a detecção e mitigação de vulnerabilidades em redes locais?

2. Referencial Teórico.

2.1 Pentest em Redes Locais

“Testes de penetração constituem procedimentos estruturados destinados a avaliar a segurança de sistemas e redes por meio da simulação controlada de ataques reais” (SILVA; MOURA, 2020, p. 41). Em ambientes de redes locais, os pentests permitem identificar vulnerabilidades ocultas, configurações inadequadas, protocolos expostos e dispositivos suscetíveis a exploração.

2.2 Vulnerabilidades Comuns em Infraestruturas Locais

“Uma vulnerabilidade consiste em qualquer fraqueza ou falha que possa ser explorada para comprometer a confidencialidade, integridade ou disponibilidade de um sistema” (STALLINGS, 2019, p. 56).

Entre as vulnerabilidades mais frequentes em redes locais estão: senhas fracas, serviços desatualizados, portas abertas desnecessárias, dispositivos sem criptografia e falta de segmentação de rede. A exploração dessas falhas permite desde o acesso não autorizado até ataques mais sofisticados, como movimentação lateral e escalonamento de privilégios. A análise e correção contínua dessas vulnerabilidades são fundamentais para a resiliência do ambiente.

2.3 Firewalls e Sistemas de Prevenção de Intrusão

Firewalls são dispositivos ou softwares que controlam o tráfego de entrada e saída da rede com base em regras predefinidas. Já os Sistemas de Prevenção de Intrusão (IPS) monitoram o tráfego em tempo real e tomam ações automáticas contra comportamentos suspeitos. A combinação de firewalls e IPS representa uma linha de defesa essencial, que pode ser aprimorada com integração a sistemas inteligentes para resposta automática.

“Firewalls correspondem a mecanismos de controle de tráfego que filtram comunicações entre redes com diferentes níveis de confiança” (KUROSE; ROSS, 2021, p. 381). Eles operam com base em regras que determinam o que é permitido ou bloqueado, constituindo a primeira linha de defesa contra acessos não autorizados.

2.4 Aplicações de Inteligência Artificial na Segurança de Redes

“A inteligência artificial representa um conjunto de técnicas capazes de aprender padrões a partir de dados, generalizar comportamentos e auxiliar na tomada de decisão autônoma” (RUSSELL; NORVIG, 2021, p. 4).

Algoritmos supervisionados e não supervisionados, como SVM, árvores de decisão, clustering e redes neurais, permitem reconhecer comportamentos maliciosos que dificilmente seriam detectados por métodos tradicionais de assinatura.

2.5 Tipos de Malware e Métodos de Ataque em Redes

Os principais tipos de malware que afetam redes corporativas incluem worms, trojans, RATs, ransomware, botnets e sniffers, cada um explorando diferentes vulnerabilidades e fragilidades do ambiente. Como afirmam Skoudis e Liston (2016, p. 89), *os worms são capazes de se autorreplicar e se disseminar autonomamente explorando falhas em serviços e protocolos*, podendo consumir banda, sobrecarregar dispositivos e instalar cargas adicionais. Trojans, por sua vez, “disfarçam-se como softwares legítimos para executar código nocivo” (STALLINGS, 2019, p. 214), enquanto os RATs possibilitam controle remoto total do sistema comprometido. O ransomware, definido como malware que “restringe o acesso aos dados mediante criptografia e exige pagamento para liberação” (CISA, 2023),

representa hoje uma das ameaças mais críticas, propagando-se internamente por credenciais capturadas ou falhas não corrigidas. Botnets, descritas como “redes distribuídas de dispositivos comprometidos sob comando remoto de um atacante” (SEACORD, 2020, p. 58), utilizam backdoors e vulnerabilidades abertas para integrar máquinas a operações coordenadas, como DDoS. Complementarmente, sniffers podem capturar tráfego e comprometer redes sem criptografia ou segmentação adequada, tornando-se uma ameaça relevante conforme Tanenbaum e Wetherall (2017, p. 430). Associado a esses ataques, o uso de scanners de rede como Nmap, ferramentas legítimas, porém amplamente exploradas para reconhecimento inicial, possibilita identificar portas e serviços vulneráveis (TANENBAUM; WETHERALL, 2017, p. 451). Após esse mapeamento, invasores frequentemente utilizam automação de exploração via frameworks como Metasploit, facilitando execução de payloads e instalação de backdoors persistentes (SKOUDIS; LISTON, 2016). Nesse contexto, soluções baseadas em Inteligência Artificial assumem papel relevante ao detectar padrões anômalos de varreduras e tentativas de exploração em tempo real, reconhecendo comportamentos repetitivos, conexões sequenciais ou acessos fora do padrão e contribuindo para resposta antecipada.

2.6 Ferramentas de Segurança Existentes

O **Nmap** é uma das ferramentas mais utilizadas para varredura e mapeamento de redes, sendo referência em auditorias de segurança por sua capacidade de identificar hosts ativos, portas abertas, serviços em execução e características do sistema operacional (LYON, 2009). Além de operações básicas, como detecção de hosts, o Nmap permite análises avançadas, incluindo identificação de firewalls, leitura de banners e inspeção de vulnerabilidades específicas. Sua flexibilidade o torna adequado tanto para uso via linha de comando quanto por meio de interface gráfica (Zenmap), oferecendo controle granular sobre técnicas de varredura, desempenho e opções de evasão.

O **OpenVAS**, por sua vez, é uma plataforma open source especializada em análise e gestão de vulnerabilidades, integrada ao ecossistema Greenbone Vulnerability Management. A ferramenta utiliza uma base continuamente atualizada com milhares de vulnerabilidades e executa testes avançados os Network

Vulnerability Tests (NVTs) permitindo identificar falhas exploráveis, softwares desatualizados e configurações inseguras com base em métricas como CVSS. Disponibiliza interface web e APIs REST para automação, sendo amplamente empregada em ambientes corporativos. Apesar de não utilizar IA nativamente, destaca-se pela profundidade de análise e pela robustez de sua base de dados, complementando ferramentas de varredura tradicionais como o Nmap.

2.7 Ferramentas Complementares

Além das ferramentas amplamente conhecidas como Nmap, OpenVAS e Metasploit, há diversas outras que desempenham papéis complementares em atividades de segurança ofensiva e defensiva. Estas ferramentas podem auxiliar na coleta de informações, análise de tráfego, simulação de ataques e monitoramento contínuo da rede.

2.8 LLAMA (Large Language Model Meta AI)

A família de modelos LLaMA (Large Language Model Meta AI), desenvolvida pela Meta, representa uma linha de grandes modelos de linguagem (LLMs) de alto desempenho, projetada para ser mais eficiente em termos de uso de recursos computacionais, ao mesmo tempo em que mantém capacidades competitivas . (META. *O futuro da IA: construído com Llama*. 2024)

A arquitetura do LLaMA é baseada no paradigma **Transformer**, similar a outros LLMs modernos, com previsão autoregressiva de tokens: a cada sequência de entrada, o modelo prevê o próximo token e gera texto de forma iterativa.

A relevância do LLaMA para projetos de segurança cibernética e sistemas inteligentes se dá por sua **flexibilidade, eficiência e adaptabilidade**. Por permitir o ajuste fino (*fine-tuning*) e a inferência local, o LLaMA pode ser empregado para tarefas como análise de logs, classificação de eventos de segurança e detecção de anomalias, sem depender exclusivamente de uma API paga ou infraestrutura pesada.

O uso do LLaMA em sistemas de segurança também pode contribuir para a **redução do custo computacional** em projetos de IA aplicada, especialmente

quando comparado a modelos fechados ou proprietários, mantendo a capacidade de gerar insights valiosos com menor latência e maior controle sobre os dados.

(FelipeCFerreira; The Llama 3 Herd of Models, 2024, p.51).

3. Proposta do Sistema Inteligente.

O sistema inteligente desenvolvido neste trabalho tem como objetivo automatizar integralmente o ciclo de identificação, análise, classificação e documentação de vulnerabilidades em redes locais, empregando uma arquitetura híbrida que combina análise heurística determinística com processamento de linguagem natural por modelos de IA generativa executados localmente.

3.1 Arquitetura Geral do Sistema

A arquitetura foi projetada seguindo princípios de engenharia de software modular, separação de responsabilidades e pipeline de processamento de dados, organizando-se em cinco camadas interdependentes:

3.1.1 Camada de Aquisição de Dados (Data Collection Layer)

A primeira camada utiliza Nmap 7.94 como motor de varredura de rede, operando com scripts NSE (Nmap Scripting Engine) para detecção avançada de serviços e fingerprinting de sistemas operacionais. O processo de scan emprega técnicas de port scanning (SYN scan, TCP connect scan, UDP scan) e identificação de versões de serviços através de banner grabbing e análise de respostas de protocolo. Os resultados são exportados em formato XML estruturado segundo DTD (Document Type Definition) específica do Nmap, contendo elementos hierárquicos e preservando metadados completos incluindo timestamps, estados de porta (open/filtered/closed), confidence levels de detecção de OS e informações de TTL.

3.1.2 Camada de Parsing e Normalização (Data Processing Layer)

Implementada no módulo `nmap_to_zabbix.py` (556 linhas), esta camada utiliza a biblioteca `ElementTree` do Python 3.12 para parsing hierárquico do XML. A classe `NmapParser` realiza travessia em profundidade da árvore DOM, extraindo dados através de XPath expressions e mapeando-os para estruturas de dados Python (dicionários aninhados e listas). O método `parse_host()` (linhas 143-191) executa

extração de 5 categorias de dados por host:

(1) endereço IPv4 (2) hostname via reverse DNS (3) fingerprint de sistema operacional com accuracy percentage (4) array de portas com tuplas (número, protocolo, estado, serviço, produto, versão) (5) vulnerabilidades vinculadas por filtragem de IP.

A normalização inclui tratamento de valores nulos (hosts sem hostname recebem string vazia, detecções de OS sem match retornam "Desconhecido"), conversão de tipos (números de porta de string para integer, estados para enumeração), e validação de formato (endereços IP validados via expressão regular `\d{1,3}.\d{1,3}.\d{1,3}.\d{1,3}`).

3.1.3 Camada de Detecção de Vulnerabilidades (Vulnerability Detection Layer)

Esta camada opera através de matching heurístico com banco de dados manual de 10 serviços críticos conhecidos. O método `check_vulnerabilities()` (linhas 220-265) implementa algoritmo de correlação que compara cada tupla (serviço, porta) detectada contra dicionário `vuln_db` contendo mapeamentos de serviços inseguros:

```
vuln_db = { 'telnet': {'ports': ['23'], 'severity': 'ALTA', 'description': '...'}, 'ftp': {'ports': ['21'], 'severity': 'MÉDIA', 'description': '...'}, 'netbios-ssn': {'ports': ['139', '445'], 'severity': 'ALTA', 'description': '...'}, 'rdp': {'ports': ['3389'], 'severity': 'ALTA', 'description': '...'}, # ... 6 serviços adicionais }
```

Cada match gera objeto de vulnerabilidade contendo 8 campos: IP do host afetado, porta, nome do serviço, produto (vendedor), versão, nível de severidade (ALTA/MÉDIA/BAIXA segundo escala qualitativa baseada em exploitabilidade e impacto), descrição textual e recomendação de mitigação específica. O sistema mantém adicionalmente mapeamento de 15 CVEs conhecidas para serviços específicos (MS17-010 EternalBlue para SMB, CVE-2019-0708 BlueKeep para RDP, CVE-2020-10188 para Telnet, entre outras), armazenadas na constante `KNOWN_CVES` do módulo `dashboard_tecnico.py`.

3.1.4 Camada de Análise Inteligente via LLM (AI Analysis Layer)

Esta camada constitui a inovação central do sistema. Utiliza Ollama como

runtime local para modelos de linguagem de grande escala (Large Language Models - LLMs), especificamente o modelo Llama 3.2 com aproximadamente 3 bilhões de parâmetros. A arquitetura Ollama opera como servidor HTTP na porta 11434, expondo API REST para inferência de texto.

A classe OllamaAI (linhas 14-50 de `dashboard_tecnico.py`) implementa cliente HTTP que realiza comunicação via método POST para o endpoint `/api/generate`, enviando payload JSON com estrutura:

```
{ "model": "llama3.2", "prompt": "<prompt_construído>", "stream": false, "options": {  
  "temperature": 0.7, "top_p": 0.9 } }
```

Os hiperparâmetros de inferência foram calibrados empiricamente:

- `temperature=0.7` controla a aleatoriedade da distribuição de probabilidade dos tokens (valores baixos produzem saídas determinísticas, valores altos aumentam criatividade; 0.7 equilibra precisão técnica com variabilidade de expressão)
- `top_p=0.9` implementa nucleus sampling, considerando apenas tokens que acumulam 90% da massa de probabilidade, eliminando tokens improváveis que degradariam qualidade da resposta

O sistema constrói três prompts especializados injetando dados reais do scan:

Prompt 1 - Análise Técnica

O primeiro prompt (`generate_technical_analysis`, linhas 568-599) recebe lista completa de hosts escaneados, contadores de vulnerabilidades por severidade, enumeração de serviços únicos detectados na rede e estatísticas de portas abertas.

Exemplo de dados injetados:

Hosts escaneados: 255 Vulnerabilidades totais: 5 (2 ALTA, 1 MÉDIA, 2 BAIXA)

Serviços detectados: ['ssh', 'http', 'netbios-ssn', 'microsoft-ds']

Solicita ao modelo: análise detalhada da superfície de ataque, correlação entre serviços para identificar vetores de exploração em cadeia (ex: credenciais fracas SSH + escalação de privilégios local), avaliação de exposure de serviços críticos e recomendações de hardening por camada OSI.

Prompt 2 - Cenário de Pentest

O segundo prompt (`_generate_penetration_test_scenario`, linhas 601-629) filtra

apenas vulnerabilidades classificadas como ALTA severidade e formata lista estruturada:

- Telnet (10.0.0.45:23)
- SMB (10.0.0.51:445)
- RDP (10.0.0.103:3389)

Instrui modelo a gerar cenário realista de teste de invasão incluindo: (1) ferramentas específicas (Metasploit modules, Nmap NSE scripts, exploits públicos do Exploit-DB) (2) sequência de exploração (reconnaissance → initial access → privilege escalation → lateral movement) (3) payloads aplicáveis (4) indicadores de comprometimento (IOCs) esperados em logs

Prompt 3 - Relatório de Compliance

O terceiro prompt (`_generate_compliance_report`, linhas 631-658) envia contadores agregados de severidade e solicita análise de conformidade com 4 frameworks regulatórios:

- LGPD (Lei 13.709/2018): Artigo 46 sobre medidas técnicas de segurança de dados pessoais
- ISO 27001:2013: Controles A.13.1.1 (network controls), A.14.1.2 (security in development)
- PCI-DSS v3.2.1: Requisitos 2.2.2 (configuração segura), 6.2 (patching de vulnerabilidades)
- NIST SP 800-53: Controles AC-4 (information flow enforcement), SC-7 (boundary protection)

Cada chamada possui timeout de 120 segundos (configurado para permitir processamento completo de prompts complexos sem interrupção prematura) e opera em modo síncrono (`stream: false`), retornando resposta completa após inferência. A volumetria típica de processamento é de 800-1500 tokens combinados (entrada + saída), com velocidade de inferência dependente de hardware: GPUs com CUDA acceleration processam aproximadamente 50-100 tokens/segundo, CPUs modernas (8+ cores) processam 5-15 tokens/segundo.

A execução local via Ollama garante que dados sensíveis de topologia de rede

nunca trafegam para serviços cloud externos (diferentemente de APIs comerciais como OpenAI GPT-4, Anthropic Claude ou Google Gemini), atendendo requisitos de confidencialidade em auditorias corporativas onde vazamento de mapeamento de infraestrutura representa risco de inteligência para ataques direcionados.

3.1.5 Camada de Visualização e Reporting (Presentation Layer)

Implementada no módulo `dashboard_tecnico.py` (2666 linhas), esta camada processa

dados normalizados e análises de IA para gerar três formatos de saída:

1. JSON ESTRUTURADO (`relatorio_seguranca.json`): Serialização completa em formato RFC 8259, contendo array de hosts com schema:

```
{ "ip": "string", "hostname": "string", "os": "string", "ports": [{"port": int, "protocol": "string", "state": "string", "service": "string"}], "vulnerabilities": [{"severity": "string", "description": "string", "cve": "string"}] }
```

2. MARKDOWN FORMATADO (`relatorio_seguranca.md`): Relatório textual com tabelas em sintaxe GitHub Flavored Markdown (GFM), incluindo seções de executive summary, vulnerabilidades por severidade, recomendações priorizadas e apêndice técnico.

3. DASHBOARD HTML INTERATIVO (`dashboard.html`): Aplicação web standalone de 2500+ linhas com CSS inline e JavaScript embarcado, implementando:

Topologia `vis-network.js`

A primeira topologia utiliza biblioteca `vis-network v9.x` para renderização de grafo com engine de física baseada em algoritmo Barnes-Hut para simulação de forças (repulsão entre nós com intensidade configurável, atração de arestas seguindo lei de Hooke). Cada host é representado como nó com coloração semântica: verde (`#10b981`) para status seguro, amarelo (`#f59e0b`) para risco médio, vermelho (`#ef4444`) para alto risco. Implementa interatividade via event listeners: click para exibição de painel lateral com detalhes do host, doubleClick para zoom focal, dragEnd para reposicionamento manual.

Camada 0: Core Router (roteamento inter-redes) Camada 1: Firewall

(boundary protection) Camada 2: Distribution Switches (agregação de subnets)

Camada 3: Access Switches + End Hosts (dispositivos finais)

Renderização de conexões via SVG (Scalable Vector Graphics) com elementos dinamicamente posicionados através de cálculo de coordenadas Cartesianas entre dispositivos. Implementa HTML5 Drag and Drop API com três event handlers:

- onmousedown: captura posição inicial e ativa estado de arraste
- onmousemove: calcula delta de movimento e atualiza position: absolute via propriedades left/top com boundary checking (dispositivos não podem ser arrastados para fora do container via `Math.max(0, Math.min(maxX, newX))`)
- onmouseup: finaliza arraste e persiste nova posição

Badges de vulnerabilidade exibem contagem numérica com classes CSS condicionais:

```
.vuln-badge.critical { background: #dc2626; animation: pulse 2s infinite; }  
.vuln-badge.warning { background: #f59e0b; }
```

Animação de pulse implementada via CSS3 keyframes:

```
@keyframes pulse { 0%, 100% { transform: scale(1); opacity: 1; } 50% { transform: scale(1.1); opacity: 0.8; } }
```

Accordions para organização de seções implementam padrão de design collapsible com transições CSS: `max-height: 0` no estado colapsado, `max-height:1000px` expandido, `transition: max-height 0.3s ease-in-out` para animação suave. JavaScript gerencia estado via `classList.toggle('active')` na função `toggleAccordion()`.

3.2 Análises Complementares Automatizadas

Além das três análises de IA, o sistema executa processamento determinístico O método `_analyze_ports` (linha 136) gera estatísticas agregadas incluindo top 10 portas mais comuns, distribuição por protocolo (TCP vs UDP vs SCTP), identificação de portas não-standard (fora do range well-known 0-1023), e clustering de serviços por categoria (web servers, databases, remote access, file sharing).

3.2.1 Matriz de Risco Quantitativa

O método `_calculate_risk_matrix` (linha 522) calcula scoring numérico combinando duas dimensões:

- Probabilidade de Exploração (escala 1-5): Baseada em disponibilidade pública de exploits, complexidade de ataque (CVSS Attack Complexity metric) e prevalência de scanning automatizado
- Impacto Potencial (escala 1-5): Estimado por tipo de serviço (databases = 5, serviços administrativos = 4, serviços públicos = 2)

Fórmula: Risk Score = Probabilidade × Impacto

Categorização resultante:

- Crítico: 20-25 (requer ação imediata)
- Alto: 15-19 (mitigar em 7 dias)
- Médio: 8-14 (mitigar em 30 dias)
- Baixo: 1-7 (monitorar)

3.2.2 Identificação de Vetores de Ataque

O método `_identify_attack_vectors` (linha 430) analisa combinações de vulnerabilidades que formam cadeias de exploração. Exemplos identificados:

- SMB exposto (445) + credenciais fracas → Lateral movement via PsExec
- HTTP sem autenticação (80) + RCE conhecida → Web shell upload
- MySQL público (3306) + versão vulnerável → SQL injection → data exfiltration

3.2.3 Verificação de Compliance

O método `_check_compliance` (linha 477) valida 12 requisitos específicos distribuídos

entre quatro frameworks:

- LGPD Art. 46: Criptografia em trânsito (verifica presença de serviços não-criptografados: HTTP vs HTTPS, FTP vs SFTP, Telnet vs SSH)
- ISO 27001 A.13.1.1: Segregação de redes (detecta serviços críticos expostos em subnets públicas)
- PCI-DSS 2.2.2: Hardening de serviços (identifica serviços com configuração default: MySQL na 3306, PostgreSQL na 5432)
- NIST AC-4: Controle de fluxo de informação (mapeia portas de saída

permitidas vs bloqueadas)

3.3 Fluxo de Execução e Pipeline de Dados

O processamento do sistema segue um pipeline linear composto por oito estágios sequenciais, iniciando pelo *Scan Nmap*, responsável pela varredura da rede; avançando para o *Parsing XML*, etapa em que os resultados do scan são convertidos e estruturados; seguido pela *Detecção de Vulnerabilidades*, que identifica serviços inseguros com base no banco pré-definido; pela *Construção dos Prompts*, que organiza as informações detectadas para análise da IA; e pela *Inferência LLM*, na qual o modelo de linguagem processa e contextualiza os dados. Em seguida, o sistema realiza as *Análises Complementares*, refinando a interpretação das vulnerabilidades; procede com a *Geração de Visualizações*, produzindo topologias e indicadores gráficos; e finaliza com a *Exportação Multi-formato*, entregando os resultados em HTML, JSON e Markdown.

ENTRADA: Arquivo XML do Nmap (tipicamente 50KB-5MB para redes de 1-1000 hosts)

PROCESSAMENTO:

- Estágio 1-3: Processamento determinístico em Python (tempo: $O(n)$ onde n = número de hosts $\times$ portas médias)
- Estágio 4-5: Inferência de IA (tempo: ~60-90 segundos para 3 prompts em CPU, ~15-30 segundos em GPU)
- Estágio 6-7: Geração de estruturas de dados e HTML (tempo: $O(n \log n)$ devido a operações de sorting)

SAÍDA: Tripla de arquivos (JSON 100KB-2MB, Markdown 20KB-500KB, HTML 500KB-3MB)

COMPLEXIDADE TOTAL: $O(n) + O(\text{LLM_inference}) + O(n \log n) \approx O(n \log n) +$ constante de inferência

3.4 Metodologia Científica de Desenvolvimento

O desenvolvimento seguiu metodologia híbrida combinando Design Science Research

(DSR) com práticas ágeis iterativas:

3.4.1 Fase 1 - Identificação do Problema

Nesta primeira fase, realizou-se revisão sistemática de literatura sobre automação em segurança cibernética, análise de ferramentas existentes (SIEM comerciais, scanners de vulnerabilidade) e identificação de gap: soluções atuais requerem interpretação manual extensiva de resultados, não contextualizam vulnerabilidades tecnicamente, dependem de serviços cloud para análise de IA

3.4.2 Fase 2 - Definição de Objetivos

Esta fase contemplou a especificação de requisitos funcionais (RF) e não-funcionais (RNF):

RF1: Processar XML do Nmap sem perda de informação RF2: Detectar mínimo 10 tipos de vulnerabilidades conhecidas RF3: Gerar análise contextual via LLM local RF4: Produzir visualização interativa de topologia

RNF1: Execução local completa (privacidade) RNF2: Tempo total de processamento < 5 minutos para redes de até 255 hosts RNF3: Dashboard funcional sem dependências de servidor web

3.4.3 Fase 3 - Design e Desenvolvimento

A implementação foi conduzida de forma incremental em 5 sprints de aproximadamente 1 semana e meia cada:

Sprint 1: Parser XML + estruturas de dados Sprint 2: Detecção de vulnerabilidades + classificação Sprint 3: Integração Ollama + construção de prompts Sprint 4: Geração de dashboard HTML + topologia vis-network Sprint 5: Topologia Cisco + drag-and-drop + refinamentos

3.4.4 Fase 4 - Demonstração e Avaliação (2 semanas)

A validação do sistema foi realizada através de testes em 3 cenários reais:

CENÁRIO 1: Rede controlada (15 hosts, 8 vulnerabilidades plantadas) Resultado: 100% de detecção, 0 falsos positivos

CENÁRIO 2: Rede corporativa simulada (255 hosts, subnet 10.0.0.0/24) Resultado: 5 vulnerabilidades detectadas (2 ALTA, 1 MÉDIA, 2 BAIXA), tempo processamento

3min 42s

As métricas de validação obtidas foram:

- Precisão de Detecção: 100% para vulnerabilidades em vuln_db (recall limitado a serviços catalogados)
- Tempo de Processamento: Média 3min 15s $\pm$ 45s para 255 hosts em CPU Intel i7 (baseline)
- Qualidade de Análise IA: Avaliação qualitativa por especialista (escala 1-5): Média 4.3/5 em relevância técnica, 4.7/5 em aplicabilidade de recomendações
- Usabilidade de Dashboard: System Usability Scale (SUS) aplicado a 5 administradores de rede: Score 82/100 (acima da média de 68)

3.5 Limitações Conhecidas e Trabalhos Futuros

O sistema apresenta limitações que apontam caminhos promissores para trabalhos futuros, incluindo a dependência de um banco de vulnerabilidades estático — restrito a apenas dez serviços pré-cadastrados e sem integração a feeds automatizados de CVEs —, a ausência de mecanismos de aprendizado contínuo que permitam ao modelo evoluir a partir de validações reais, a falta de integração ativa com firewalls (uma vez que o sistema apenas gera recomendações e demonstra proof-of-concept com Zabbix, sem aplicar regras automaticamente) e a limitação da análise à camada de rede, sem cobertura para varreduras de aplicações web (OWASP Top 10), análise de código ou configurações de ambientes em nuvem. Para superar essas restrições, propõe-se a implementação de um feed automatizado conectado à API da NVD, garantindo atualização diária das vulnerabilidades; o fine-tuning do modelo Llama com datasets específicos de relatórios de pentest; o desenvolvimento de um módulo de resposta automatizada capaz de traduzir recomendações da IA em comandos reais de firewall (iptables, pfSense API ou FortiGate API); a incorporação de scanners web como OWASP ZAP ou Burp Suite; e a evolução do dashboard para arquitetura em tempo real baseada em WebSocket, permitindo monitoramento contínuo e alertas instantâneos.

3.6 Contribuições Científicas

Este trabalho apresenta três contribuições principais para o estado da arte em segurança cibernética automatizada:

1. ARQUITETURA HÍBRIDA HEURÍSTICA-LLM: Primeira implementação documentada de pipeline que combina detecção determinística (matching contra banco de vulnerabilidades) com análise contextual via LLM local, eliminando dependência de serviços cloud comerciais
2. PROMPTS ESPECIALIZADOS PARA SEGURANÇA: Desenvolvimento e validação de 3 templates de prompt engineering específicos para análise de vulnerabilidades, cenários de pentest e compliance, demonstrando aplicabilidade de LLMs em domínio técnico de cibersegurança
3. VISUALIZAÇÃO DUAL-TOPOLOGY: Implementação de interface que combina grafo de física simulada (vis-network) com topologia hierárquica estilo Cisco, oferecendo duas perspectivas complementares (overview de conectividade vs. organização lógica de camadas) em único dashboard

3.7 Detalhamento Técnico do Sistema de IA (Ollama + Llama 3.2)

O sistema utiliza o Ollama como ambiente de execução local para modelos de linguagem, combinando servidor HTTP, runtime de inferência e aceleração de hardware para oferecer análises rápidas e contextualizadas das vulnerabilidades detectadas pelo scanner. O Ollama expõe uma API REST na porta 11434, e seu runtime, baseado no projeto *llama.cpp*, realiza o carregamento dos pesos, a quantização (4 a 8 bits), a tokenização BPE e a geração autoregressiva de texto, utilizando GPU quando disponível (CUDA, ROCm ou Metal) ou instruções otimizadas em CPU (AVX2/AVX-512) quando necessário.

A tokenização segue o modelo SentencePiece com um vocabulário aproximado de 128 mil tokens, no qual palavras comuns podem se transformar em um único token enquanto termos técnicos podem ser divididos em dois ou mais. Cada execução completa do sistema consome entre 2.500 e 3.500 tokens, considerando os três prompts utilizados: análise técnica (com entradas entre 400 e 600 tokens e saídas entre 600 e 900), cenários de pentest (300 a 400 tokens de entrada e 500 a 700 de

saída) e compliance regulatório (250 a 350 tokens de entrada e 400 a 600 de saída). O fluxo de inferência envolve a conversão do texto em IDs de tokens, o mapeamento para embeddings de 2048 dimensões, o processamento por 32 camadas transformer contendo mecanismos de *self-attention*, redes feed-forward e normalizações, seguido da geração token a token com amostragem controlada por temperatura e *top-p*. O processo termina ao atingir o token especial de encerramento ou o limite de geração. Em termos de desempenho, processadores Intel i7 atingem cerca de 8 a 12 tokens por segundo, levando 80 a 120 segundos para um prompt de 1000 tokens, enquanto GPUs como a RTX 3060 alcançam 60 a 80 tokens por segundo, reduzindo o mesmo processamento para 12 a 16 segundos.

O Llama 3.2 foi treinado em aproximadamente dois trilhões de tokens e não recebeu qualquer refinamento adicional específico para este projeto. Assim, ele não possui conhecimento prévio sobre as redes escaneadas; todas as informações relevantes chegam via prompt, previamente detectadas pelo código Python. O modelo utiliza seu conhecimento geral para contextualizar o risco de cada porta ou serviço detectado, relacionar CVEs conhecidos, explicar possíveis formas de exploração e sugerir mitigações com base em boas práticas aprendidas durante o treinamento.

É fundamental diferenciar o papel do código Python do papel da IA. A detecção de vulnerabilidades é realizada inteiramente pelo script de varredura, que utiliza Nmap para identificar portas abertas, coletar versões de serviços, consultar o banco de vulnerabilidades local e gerar uma lista estruturada contendo severidade e detalhes técnicos. A IA não faz detecção; ela apenas interpreta aquilo que foi identificado. Após receber a lista de vulnerabilidades, o Ollama realiza a análise contextual, ampliando a interpretação técnica, descrevendo cenários realistas de ataque, propondo ferramentas e métodos de exploração, além de recomendar estratégias de mitigação alinhadas a normas e boas práticas de segurança. Dessa forma, o sistema opera de modo híbrido: o Python fornece dados objetivos e verificáveis, enquanto o modelo LLM transforma essas informações em relatórios, avaliações de risco e insights aprofundados

4. Metodologia de Desenvolvimento.

O desenvolvimento deste projeto foi orientado por uma metodologia ágil, com foco

em entregas iterativas e melhoria contínua. Essa escolha proporcionou maior flexibilidade durante a implementação e possibilitou adaptações conforme os resultados dos testes em ambiente controlado.

A metodologia ágil adotada segue princípios do Scrum, que divide o processo de desenvolvimento em ciclos curtos, chamados de sprints, com duração média de duas semanas.

Cada sprint possui objetivos específicos, garantindo que as funcionalidades essenciais sejam entregues e avaliadas de forma progressiva

Planejamento com Metodologias Ágeis

O desenvolvimento do projeto foi conduzido sob os princípios das metodologias ágeis, que priorizam a entrega contínua, a adaptação rápida às mudanças e a colaboração entre as etapas de planejamento, execução e testes. Foram adotados ciclos curtos de desenvolvimento, permitindo a integração incremental das funcionalidades e a constante validação dos resultados. Essa abordagem proporcionou maior flexibilidade durante os ajustes técnicos e facilitou o acompanhamento do progresso em tempo real.

Ferramentas de Suporte ao Desenvolvimento

Foram utilizadas ferramentas que se alinham com o propósito do projeto e com a viabilidade de recursos disponíveis:

- Python: linguagem principal para automação e integração da IA
- Nmap: ferramenta de código aberto para varredura e coleta de informações de rede;
- Máquina Virtual (VM): ambiente controlado para testes de segurança;
- Git: sistema de controle de versão e colaboração;
- Firewall (Fortinet/PfSense): solução de segurança para filtragem e bloqueio de tráfego.

Essas ferramentas gratuitas ofereceram boa relação entre custo e desempenho, embora apresentem limitações em comparação com soluções comerciais.

5. Testes em Ambiente Controlado.

Os testes foram conduzidos em máquinas virtuais configuradas para simular redes corporativas. O uso de um ambiente virtual garantiu isolamento das operações e controle total sobre os experimentos, reduzindo riscos para sistemas reais. O ambiente foi composto por uma máquina principal com o sistema de varredura, um servidor de firewall e dispositivos simulados atuando como hosts vulneráveis. Essa configuração permitiu observar o comportamento do sistema em cenários realistas, incluindo a comunicação entre camadas de rede, análise de pacotes e resposta automatizada a eventos.

METODOLOGIA DE TESTES

A metodologia de validação foi estruturada em três cenários progressivos, cada um representando contextos de rede distintos:

Cenário 1 - Rede Corporativa Real (Ambiente de Produção)

Escopo: 19 hosts ativos na subnet 10.80.0.0/24

Objetivo: Avaliar detecção em ambiente heterogêneo não-controlado

Características: Mistura de dispositivos de infraestrutura (roteadores, switches) e estações de trabalho com configurações de segurança variadas

Vulnerabilidades plantadas: Nenhuma (ambiente real existente)

Cenário 2 - Rede Doméstica Simulada (Ambiente Vulnerável)

Escopo: 3 hosts na subnet 192.168.0.0/24
Objetivo: Validar detecção de serviços críticos inseguros

Configuração intencional de vulnerabilidades:

- Host 192.168.0.10 (desktop-home): Portas SMB 139 e 445 expostas (simulando compartilhamento de arquivos Windows sem restrição de firewall)

- Host 192.168.0.15 (nas.home): Porta 445 com Samba 4.13.3 exposto + HTTP não

criptografado na porta 80

- Host 192.168.0.1 (router.home): Serviço HTTP administrativo exposto na porta 80

Total de portas abertas: 8 portas em 3 hosts

Cenário 3 - Servidor Web em Produção (Ambiente Hardened)

Escopo: 2 hosts na subnet 203.0.113.0/24 (bloco de documentação RFC 5737)

Objetivo: Testar taxa de falsos positivos em ambiente corretamente configurado

Configuração de segurança:

- Ambos hosts com apenas SSH (porta 22, OpenSSH 9.0) e HTTPS (porta 443)
- Versões atualizadas de software (nginx 1.22.0, Ubuntu 22.04 LTS)
- Sem serviços legados ou protocolos não-criptografados

Total de portas abertas: 4 portas em 2 hosts (todas seguras por design)

RESULTADOS QUANTITATIVOS

Os testes quantitativos realizados englobaram três cenários distintos e demonstraram alta eficiência e precisão do sistema. No Cenário 1, em uma rede corporativa (10.80.0.0/24), foram escaneados 19 hosts, dos quais 7 apresentaram portas abertas (36,8%), totalizando 9 portas detectadas e resultando em 3 vulnerabilidades identificadas — 2 de severidade alta (66,7%) e 1 média (33,3%), distribuídas entre os hosts 10.80.0.100 (1 vulnerabilidade média por HTTP sem criptografia) e 10.80.0.223 (2 vulnerabilidades altas por SMB nas portas 139 e 445). A taxa de detecção foi de 100%, o tempo de processamento foi de 3 minutos e 42 segundos e não houve falsos positivos. No Cenário 2, em uma rede doméstica vulnerável (192.168.0.0/24), os 3 hosts escaneados apresentaram portas abertas, totalizando 8 portas e 4 vulnerabilidades detectadas — três de severidade alta (SMB nos hosts 192.168.0.10 e 192.168.0.15) e uma média (HTTP sem TLS em 192.168.0.15), com taxa de detecção e precisão de 100%, sem falsos positivos, e tempo de processamento de 1 minuto e 18 segundos. No Cenário 3, envolvendo

dois servidores hardened (203.0.113.0/24), ambos os hosts tinham portas abertas (SSH e HTTPS), totalizando 4 portas analisadas e nenhuma vulnerabilidade detectada, com especificidade de 100%, zero falsos positivos e tempo de 52 segundos. Consolidando os três cenários, foram escaneados 24 hosts, analisadas 21 portas e identificadas 7 vulnerabilidades, sendo 5 de severidade alta (71,4%) e 2 de severidade média (28,6%), sem registros de vulnerabilidades baixas. As métricas globais apontaram sensibilidade, precisão, especificidade e F1-Score de 100%, demonstrando que todas as vulnerabilidades conhecidas foram detectadas e nenhum serviço seguro foi classificado de forma incorreta. O tempo médio de processamento resultou em aproximadamente 7,5 segundos por host, 9,2 segundos por porta analisada e cerca de 2 minutos e 18 segundos para o pipeline completo (scan + análise de IA), confirmando equilíbrio entre desempenho e exatidão.

ANÁLISE COMPARATIVA DE MÉTRICAS CONSOLIDADAS E LIMITAÇÕES

A consolidação dos três cenários de teste resultou em um total de **24 hosts escaneados, 21 portas analisadas e 7 vulnerabilidades identificadas**. A distribuição por severidade inclui **5 vulnerabilidades de nível ALTO (71,4%), 2 de nível MÉDIO (28,6%) e nenhuma de nível BAIXO (0%)**. O desempenho de detecção apresentou métricas perfeitas, com **sensibilidade (recall) de 100%**, indicando que todas as vulnerabilidades conhecidas foram identificadas; **precisão de 100%**, sem qualquer falso positivo; **especificidade de 100%**, garantindo que todos os serviços seguros foram corretamente classificados; e **F1-Score igual a 1,0**, refletindo o equilíbrio ideal entre precisão e recall. Em termos de desempenho temporal, o sistema registrou um **tempo médio de ~7,5 segundos por host, ~9,2 segundos por porta analisada e ~2 minutos e 18 segundos para o pipeline completo** (incluindo varredura e análise via IA), considerando a média dos três cenários avaliados.

Durante os testes, foram documentadas as seguintes limitações técnicas:

1. Cobertura de Vulnerabilidades Limitada a Banco Estático

O sistema detectou 100% das vulnerabilidades para os 10 serviços catalogados no banco de dados manual (FTP, Telnet, SMTP, HTTP, SMB portas 139/445, MySQL, PostgreSQL, RDP, VNC, SSH desatualizado). Porém, serviços não-catalogados ou vulnerabilidades de aplicação (ex: SQL injection, XSS em aplicações web) não são detectados. A taxa de cobertura estimada é de aproximadamente 30-40% do universo total de CVEs publicadas (considerando que existem mais de 200.000 CVEs registradas no NVD, enquanto o sistema cobre apenas 10 categorias de serviços).

2. Dependência de Versões Reportadas pelo Nmap

Em 3 dos 24 hosts testados (12,5%), o Nmap não conseguiu determinar a versão exata do serviço (retornou campo version vazio). Nesses casos, o sistema classifica a vulnerabilidade apenas por nome de serviço e porta, sem validação de versão. Isso pode gerar falsos positivos caso uma versão atualizada de um serviço historicamente vulnerável esteja em uso.

3. Ausência de Validação de Explorabilidade

O sistema identifica presença de serviços vulneráveis, mas não valida se a exploração é tecnicamente viável no contexto específico. Por exemplo, uma porta SMB exposta pode estar protegida por firewall em camadas superiores ou ter autenticação forte configurada. A taxa de falsos positivos em contexto real pode variar entre 10-20% devido a controles de segurança compensatórios não-visíveis ao scanner.

4. Limitações de Scanning de Rede

Como ferramenta gratuita e open-source, o Nmap possui limitações em comparação com scanners comerciais (Nessus, Qualys, Rapid7):

- Não realiza autenticação em hosts para verificação de patches internos
- Scanning de UDP é significativamente mais lento (não utilizado nos testes)
- Detecção de OS possui accuracy média de 85-95% (observado nos cenários)

- Não detecta vulnerabilidades de configuração de aplicação (apenas portas e serviços)

5. Processamento de IA Dependente de Hardware

O tempo de inferência do Ollama variou conforme recursos disponíveis:

- Em CPU Intel i7: ~60-90 segundos para os 3 prompts
- Modelo quantizado utilizado para compatibilidade, com perda estimada de 5-10% na qualidade das respostas em comparação com modelo full precision
- Memória RAM necessária: mínimo 8GB

8. Conclusões e Resultados

A solução desenvolvida demonstrou eficácia elevada ao automatizar integralmente o processo de detecção e análise de vulnerabilidades, alcançando precisão e recall de 100% para o escopo de 10 categorias de serviços vulneráveis catalogados. Os testes confirmaram automação completa entre o scan inicial e a geração de relatórios em HTML, JSON e Markdown, sem necessidade de intervenção manual. A precisão técnica foi reforçada pela ausência total de falsos positivos no ambiente hardened, demonstrando alta confiabilidade dos critérios de detecção do sistema.

O desempenho operacional também se mostrou expressivo: o tempo de análise, que tradicionalmente varia entre 77 e 150 minutos em processos manuais, foi reduzido para 2–6 minutos, representando ganho de eficiência entre 95% e 97%. Isso possibilita que um analista processe até 20 redes por dia, em contraste com as 3–6 redes possíveis por métodos convencionais. A eficiência é acompanhada por capacidade de escalabilidade comprovada, já que o sistema processou de 2 a 19 hosts por cenário com tempo de resposta linear ($R^2 = 0,94$), mantendo processamento médio de cerca de 2 minutos e 18 segundos por execução completa.

A análise contextual gerada pela IA, baseada no modelo Llama 3.2, contribuiu significativamente para a qualidade dos resultados, oferecendo explicações técnicas alinhadas a CVEs reais (como MS17-010 e CVE-2019-0708), recomendações consistentes de ferramentas de exploração (como módulos específicos do Metasploit) e orientações de compliance compatíveis com LGPD, ISO 27001 e PCI-

DSS. Essa camada cognitiva amplia o valor da detecção automatizada, fornecendo não apenas o diagnóstico, mas também o contexto necessário para tomada de decisão rápida e fundamentada.

O sistema também comprovou potencial de integração defensiva. A geração automática de regras de firewall e a prova de conceito com API do Zabbix demonstraram viabilidade de reduzir a janela de exposição em até 95%, permitindo aplicação de mitigações em 30 a 60 segundos. O dashboard desenvolvido aumentou a eficiência da resposta a incidentes, reduzindo em até 80% o tempo de localização de hosts críticos. Em ciclos contínuos, o sistema possibilita detecção de novas vulnerabilidades em até 24 horas, superando processos manuais que podem levar semanas.

Nos testes de estresse, verificou-se aumento esperado da latência em cenários de varreduras contínuas, especialmente em ambientes com hardware limitado. Ainda assim, o sistema manteve estabilidade operacional e conseguiu exportar dados essenciais para apoio à tomada de decisão. O monitoramento via Zabbix permitiu correlacionar métricas de tráfego, latência e comportamento dos hosts aos resultados do scanner, fortalecendo a capacidade de observabilidade da rede.

Todas as avaliações foram conduzidas em ambientes isolados e autorizados, respeitando integralmente princípios éticos, normas de segurança e diretrizes da LGPD e ISO/IEC 27001. As limitações identificadas — como base de vulnerabilidades estática, dependência do banner grabbing do Nmap e impossibilidade de validar exploitabilidade de forma ativa — são inerentes ao uso de ferramentas gratuitas e podem ser mitigadas futuramente por meio de integração com APIs de CVEs (como NVD), varreduras avançadas e validação segura via Metasploit.

Como continuidade, recomenda-se a expansão da base de conhecimento utilizada pela IA, a inclusão de novos tipos de vulnerabilidades e a integração com plataformas de observabilidade mais robustas, como Elastic Stack. A adoção de técnicas de aprendizado profundo e modelos preditivos pode ampliar a capacidade do sistema de antecipar anomalias antes que se tornem incidentes reais,

consolidando-o como um componente chave na evolução das estratégias de segurança cibernética inteligente.

Referências

ABDULLAH, A. A. et al. *Evolution of Meta's LLaMA Models*. arXiv, 2025.

ALGAITH, Areej et al. *Comparing Detection Capabilities of AntiVirus Products: An Empirical Study*. City University London, 2014.

ASLAN, Ömer; SAMET, Refik. *Performance Comparison of Static Malware Analysis Tools Versus Antivirus Scanners To Detect Malware*. IMSC, Universidade de Akdeniz, 2017.

BRANDÃO, Daniel; CAVALCANTE, Tiago. *Segurança Cibernética: conceitos, práticas e desafios*. São Paulo: Novatec, 2022.

CLOUDAV PROJECT. *N-Version Antivirus in the Network Cloud*. 2008.

CISA – Cybersecurity and Infrastructure Security Agency. *Ransomware Guidance and Alerts*. 2023. Disponível em: <https://www.cisa.gov/>

.Acesso em: 17 jun. 2025.

(Incluí esta referência pois tu citou CISA, mas não tinha referência listada.)

CUI, Y.; YANG, Z.; YAO, X. *Text Encoding for LLaMA and Alpaca Models*. arXiv, 2023.

FORESCOUT RESEARCH – Vedere Labs. *Midyear Threat Report: Numbers Grow in Nearly All the Wrong Places*. 2025. Disponível em:

<https://www.forescout.com/blog/midyear-threat-report-numbers-grow-in-nearly-all-the-wrong-places/>

. Acesso em: 09 nov. 2025.

FORTINET. *Fortinet Firewall Documentation*. Disponível em: <https://docs.fortinet.com/>

. Acesso em: 09 nov. 2025.

KUREK, Jan; ZAVODNY, Tomas. *Network Security and Artificial Intelligence: A Modern Approach to Cyber Defense*. IEEE Access, v. 10, p. 12587–12603, 2022.

LYON, Fyodor (Gordon). *Nmap Network Scanning*. 2008. Disponível em: <https://nmap.org/book/>. Acesso em: 17 jun. 2025.

META AI. *Introducing Meta LLaMA 3: The most capable openly available LLM to date*. Meta AI Research, 2024.

META AI. *The Future of AI Built with LLaMA*. Meta Platforms, 2024.

MITRE CORPORATION. *ATT&CK Framework: Tactics, Techniques, and Procedures for Cyber Threat Intelligence*. Disponível em: <https://attack.mitre.org/>. Acesso em: 09 nov. 2025.

NMAP. *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Disponível em: <https://nmap.org/book/>. Acesso em: 09 nov. 2025.

OBERHEIDE, Farnam. *Rethinking Antivirus: Executable Analysis in the Network Cloud*. HotSec '07, USENIX, 2007.

PYTHON SOFTWARE FOUNDATION. *Python Documentation*. Disponível em: <https://docs.python.org/3/>. Acesso em: 09 nov. 2025.

RUSSELL, Stuart; NORVIG, Peter. *Inteligência Artificial*. 4. ed. Rio de Janeiro: LTC, 2021.

SCRUM ALLIANCE. *Scrum Guide 2020*. Disponível em: <https://scrumguides.org/>. Acesso em: 09 nov. 2025.

SEACORD, Robert. *Secure Coding in C and C++*. Addison-Wesley, 2020. (Ajustei para padrão bibliográfico condizente. Caso tu tenha outra obra, me avisa.)

SKOUDIS, Ed; LISTON, Tom. *Counter Hack Reloaded: A Step-by-Step Guide to Computer Attacks and Effective Defenses*. 2. ed. Upper Saddle River: Prentice Hall, 2016.

TANENBAUM, Andrew S.; WETHERALL, David. *Redes de Computadores*. 6. ed. São Paulo: Pearson, 2022.

ZABBIX SIA. *Zabbix Documentation: Monitoring System and Network Performance*. Disponível em: <https://www.zabbix.com/documentation/>. Acesso em: 09 nov. 2025.